TAM Intern Task: Agentic Ticket Triage

Expected effort: 6–8 hours **Deadline: as stated in your invitation email**

Context

Our support and Technical Account Management (TAM) teams handle thousands of procurement-software tickets a year. A good support engineer does not triage a ticket from its text alone. They check who the customer is, look at account health and ticket history, find the right knowledge-base (KB) article, apply support policy, and then decide.

Build an agent that does this for one ticket at a time.

What you build

Implement `triage(ticket, ctx)` in `tam_agent/agent.py`. For each ticket, the agent must:

1. Resolve the account. `account_id` is sometimes blank.
2. Gather the account context that matters. Tier, renewal date, usage trend, open escalations, related tickets.
3. Retrieve the relevant KB articles, including support policy.
4. Decide. Category, priority, routing, escalation, TAM notification, duplicate status, and whether clarification is needed.
5. Draft a customer reply and an internal note.
6. Record every step in `reasoning_trace`.

The output format is fixed. See `tam_agent/schema.py`.

Must-have: the agent decides what to do next

Expose your lookups as tools. The LLM chooses which tool to call, reads the result, decides whether it needs more information, and decides when to stop.

> A fixed sequence of LLM calls that runs the same steps for every ticket does not meet this bar. Some tickets need one lookup. Others need three or four, including following one article to another.

Scale

The data is production-sized:

Data	Volume
Tickets	~3,000 (12 months)
Accounts	200
Usage metrics	12 months per account
KB articles	~390 (~100k tokens)

- Design for the full dataset. Do not paste the whole KB or ticket table into a prompt.

- You only need to run the 10 tickets in `data/review_set.txt`. Running a larger sample is optional.
- The data is realistic: blank fields, inconsistent labels, similar company names, outdated articles. Foreign keys are valid. See `data/README.md`.
- Review tickets reflect real situations. Some are not what they first appear.

What is provided

Starter repository: [repo link provided in your invitation email]

File	Purpose
`tam_agent/data.py`, `tam_agent/kb.py`	Loaders only. No joins or search.
`tam_agent/llm.py`	Minimal client for Ollama (free, local), Anthropic or OpenAI
`tam_agent/schema.py`	Required output format and validator
`tam_agent/agent.py`	Your code goes here. Add any modules you need.
`run.py`	Runs the agent and writes `outputs/`
`scripts/check_outputs.py`	Confirms your review-set outputs are valid
`scripts/validate_data.py`	Confirms dataset integrity

You can use any framework (LangGraph, LlamaIndex, etc.) or none. You can replace any provided file except `schema.py`.

Write SOLUTION.md (1 page max)

1. **Design.** How the agent works, which tools it has, how it decides to stop. A diagram is optional.
2. **Evaluation.** 5–8 bullets: how you would measure triage quality before putting this in production.
3. **Scale.** How does your approach behave at 10x the data (30,000 tickets, 4,000 articles)? Cover cost, latency and retrieval quality. What would you change?
4. **Limitations.** What does not work yet.
5. **How to run.**

Quick start

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env            # choose provider and model
python scripts/validate_data.py
python run.py --set review      # after implementing agent.py
python scripts/check_outputs.py
```

Submission

- Push to a GitHub repository shared with the hiring team.
- Commit `outputs/` for the 10 review tickets, plus `outputs/run_log.jsonl`.
- `python scripts/check_outputs.py` must show 10/10 valid. Submissions that fail this check are not reviewed.
- Commit as you work. Do not squash into a single commit.
- Never commit API keys or `.env`.

Rules

- Any LLM provider is fine, including a free local model. We evaluate design and reasoning, not model size.
- AI coding assistants are allowed. You must be able to explain every part of your code.

How you will be evaluated

Area	Weight
Working build	30%
Agentic depth	30%
Retrieval and decision accuracy (review set)	20%
Written reasoning (SOLUTION.md)	20%

Shortlisted candidates are invited to a 15-minute code walkthrough.